

PHAN VĂN HIẾU

HE190773

Mục tiêu (Objective)

- Làm quen với công cụ WinDbg.
- Quan sát các module được nạp trong Windows Kernel.
- Xem dữ liệu bộ nhớ của nhân hệ điều hành.
- Tìm kiếm và phân tích các hàm trong Ntoskrnl.
- Thực hành disassembly một hàm hệ thống.
- Xem cấu trúc dữ liệu của driver trong kernel.

Môi trường thực hiện (Environment)

Thành phần	Giá trị
OS	Windows 10 x64
Tool	LiveKD
Debugger	WinDbg/KD
Symbol Server	Microsoft Symbol Server

PHẦN THỰC HÀNH

Bước 1: Khởi động LiveKD

Mở Command Prompt với quyền Administrator và chạy:

```
livekd.exe -k "C:\Program Files (x86)\Windows Kits\10\Debuggers\x64\kd.exe"
```

Mô tả

LiveKD tạo một bản dump của hệ thống đang chạy và mở WinDbg/KD để phân tích kernel mà không cần khởi động hệ thống ở chế độ debug.

```
Administrator: Command Prompt - livekd.exe -k "C:\Program Files (x86)\Windows Kits\10\Debuggers\x64\kd.exe"
Microsoft (R) Windows Debugger Version 10.0.17763.7010 AMD64
Copyright (c) Microsoft Corporation. All rights reserved.

Loading Dump File [C:\Windows\livekd.dmp]
Kernel Complete Dump File: Full address space is available

Comment: 'LiveKD live system view'

***** Path validation summary *****
Response                               Time (ms)      Location
Deferred                               srv*c:\Symbols*http://msdl.microsoft.com/download/symbols
Symbol search path is: srv*c:\Symbols*http://msdl.microsoft.com/download/symbols
Executable search path is:
Windows 8 Kernel Version 9200 UP Free x64
Product: WinNt, suite: TerminalServer SingleUserTS Personal
Built by: 19041.1.amd64fre.vb_release.191206-1406
Machine Name:
Kernel base = 0xfffff805`32200000 PsLoadedModuleList = 0xfffff805`32e2a420
Debug session time: Sun Jun 28 20:30:39.300 2026 (UTC + 7:00)
System Uptime: 0 days 1:34:46.659
Loading Kernel Symbols
.....
.....
.....
Loading User Symbols
.....
Loading unloaded module list
.....
kd>
```

Bước 2: Liệt kê các module kernel

Thực hiện:

lm

Mô tả

Lệnh **lm** (List Modules) hiển thị tất cả các module và driver hiện đang được nạp trong bộ nhớ.

Kết quả

Quan sát module:

nt

Đây là module:

ntoskrnl.exe

là nhân chính của hệ điều hành Windows.

```

kd> lm
*** ERROR: Module load completed but symbols could not be loaded for LiveKd.SYS
start      end                module name
00007ff6`69c30000 00007ff6`69c63000  kd_exe      (deferred)
00007ffa`68cd0000 00007ffa`69402000  dbgeng      (deferred)
00007ffa`95500000 00007ffa`956f3000  dbghelp     (deferred)
00007ffa`a0420000 00007ffa`a048c000  symsrv      (deferred)
00007ffa`a04e0000 00007ffa`a0589000  dbgmodel    (deferred)
00007ffa`a9710000 00007ffa`a971c000  Secur32     (deferred)
00007ffa`aecda000 00007ffa`aecda000  version     (deferred)
00007ffa`b12a0000 00007ffa`b12d6000  XmlLite     (deferred)
00007ffa`b3360000 00007ffa`b33fe000  uxtheme     (deferred)
00007ffa`b3830000 00007ffa`b3842000  kernel_appcore (deferred)
00007ffa`b5830000 00007ffa`b5862000  SSPICLI     (deferred)
00007ffa`b5b20000 00007ffa`b5c20000  ucrtbase    (deferred)
00007ffa`b5c20000 00007ffa`b5d39000  gdi32full   (deferred)
00007ffa`b5d40000 00007ffa`b5dc2000  bcryptPrimitives (deferred)
00007ffa`b5dd0000 00007ffa`b60c6000  KERNELBASE  (deferred)
00007ffa`b60d0000 00007ffa`b60f2000  win32u      (deferred)
00007ffa`b6100000 00007ffa`b6127000  bcrypt      (deferred)
00007ffa`b6130000 00007ffa`b628d000  CRYPT32     (deferred)
00007ffa`b6290000 00007ffa`b632d000  msvcrt_win  (deferred)
00007ffa`b6330000 00007ffa`b64d1000  USER32     (deferred)
00007ffa`b64e0000 00007ffa`b657f000  sechost     (deferred)
00007ffa`b6610000 00007ffa`b673b000  ole32       (deferred)
00007ffa`b6800000 00007ffa`b68b1000  advapi32    (deferred)
00007ffa`b6ef0000 00007ffa`b6f9d000  shcore      (deferred)

```

Nhận xét

Qua lệnh này có thể xác định địa chỉ nạp của các driver và kernel modules.

Bước 3: Xem dữ liệu bộ nhớ của Ntoskrnl

Thực hiện:

dd nt

```

fffff805`36ba0000 fffff805`36bb1000  hwpolicy.sys
kd> dd nt
fffff805`32200000 00905a4d 00000003 00000004 0000ffff
fffff805`32200010 000000b8 00000000 00000040 00000000
fffff805`32200020 00000000 00000000 00000000 00000000
fffff805`32200030 00000000 00000000 00000000 00000118
fffff805`32200040 0eba1f0e cd09b400 4c01b821 685421cd
fffff805`32200050 70207369 72676f72 63206d61 6f6e6e61
fffff805`32200060 65622074 6e757220 206e6920 20534f44
fffff805`32200070 65646f6d 0a0d0d2e 00000024 00000000

```

Mô tả

Lệnh **dd** (Display Doubleword) hiển thị dữ liệu bộ nhớ ở dạng hexadecimal.

Nhận xét

Dữ liệu hiển thị là những byte đầu tiên của file Ntoskrnl trong bộ nhớ.

Bước 4: Xem dữ liệu dạng ASCII

Thực hiện:

da nt

```
kd> da nt
fffff805`32200000  "MZ."
kd>
```

Mô tả

Lệnh **da** hiển thị bộ nhớ ở dạng ký tự ASCII.

Kết quả

Xuất hiện:

MZ

Giải thích

"MZ" là chữ ký của tất cả các file PE (Portable Executable) trên Windows.

Bước 5: Hiển thị thông điệp DOS

Thực hiện:

da nt+4c

```
kd> da nt+4c
fffff805`3220004c  ".!This program cannot be run in "
fffff805`3220006c  "DOS mode....$"
kd> da nt+4c
fffff805`3220004c  ".!This program cannot be run in "
fffff805`3220006c  "DOS mode....$"
```

Kết quả

This program cannot be run in DOS mode

Giải thích

Thông điệp này nằm trong DOS Stub của file PE.

Bước 6: Tìm kiếm các hàm trong Kernel

Thực hiện:

x nt!*

Mô tả

Lệnh **x** dùng để liệt kê các symbol.

Kết quả

Hiển thị toàn bộ hàm và biến của Ntoskrnl.

```

fffff805`32beaef0 nt!VfZwQueryFullAttributesFile (<no parameter info>)
fffff805`32705bb0 nt!IoRaiseInformationalHardError (<no parameter info>)
fffff805`3222f4e8 nt!POP_ETW_EVENT_COMPONENT_IDLE_CONSTRAINTS = <no type information>
fffff805`329d5c80 nt! ?? ::NNGAKEGL::`string' (<no parameter info>)
fffff805`32f2f290 nt!SepAtomSd = <no type information>
fffff805`3258be54 nt!TopUnmaskInterrupt (<no parameter info>)
fffff805`327a2470 nt!ViRaiseIrqlToDpcLevel (<no parameter info>)
fffff805`325f3bd0 nt!MiGetProtectionFromPte (<no parameter info>)
fffff805`32331658 nt!_IMPORT_DESCRIPTOR_PSHED = <no type information>
fffff805`3220a4a8 nt!DEVPKEY_Device_DriverDesc = <no type information>
fffff805`32704cc0 nt!KeCapturePersistentThreadState (<no parameter info>)
fffff805`32e193e0 nt!ExExternalBootSupportInitializationEvent = <no type information>
fffff805`329db7f0 nt! ?? ::NNGAKEGL::`string' (<no parameter info>)
fffff805`329dae70 nt! ?? ::NNGAKEGL::`string' (<no parameter info>)
fffff805`32985a58 nt!BiReleasePrivilege (<no parameter info>)
fffff805`32e4bf50 nt!HalpDmaPdoListLock = <no type information>
fffff805`32a66e90 nt!HalpLbrStopRecording (<no parameter info>)
fffff805`32adca00 nt!ObOpenObjectByPointerWithTag (<no parameter info>)
fffff805`32614770 nt! ?? ::FNODOBFM::`string' (<no parameter info>)
fffff805`325b685c nt!RtlApplyImportRelocationToImage (<no parameter info>)
fffff805`326bba40 nt!HalEnumerateEnvironmentVariablesEx (<no parameter info>)
fffff805`32c979d8 nt! ?? ::EKOMKFNL::`string' = <no type information>
fffff805`3283c8c0 nt!CcPinRead (<no parameter info>)
fffff805`32c98f18 nt!CmpFlagsString = <no type information>
fffff805`32e44c10 nt!IopNumberDeviceNodes = <no type information>
fffff805`326f6e90 nt!HvlSvmSetDeviceEnabled (<no parameter info>)
fffff805`32b74c6c nt!CmDeleteCommonClassRegKey (<no parameter info>)

```

Tiếp tục:

x nt!*Create*

Mục đích

Tìm tất cả hàm chứa từ khóa:

Create

Tiếp tục:

x nt!*CreateFile*

Kết quả

```

fffff805`32af6d38 nt!PopDirectedDripsDiagCreateBlockerEntryULong (<no parameter info>)
kd> x nt!*CreateFile*
fffff805`32f420a0 nt!pXdvIoCreateFileSpecifyDeviceObjectHint = <no type information>
fffff805`32b6fb50 nt!BiCreateFileDeviceElement (<no parameter info>)
fffff805`3280b890 nt!NtCreateFile (<no parameter info>)
fffff805`3280b6d0 nt!IoCreateFileEx (<no parameter info>)
fffff805`325fa510 nt!ZwCreateFile (<no parameter info>)
fffff805`32bd28f0 nt!VerifierIoCreateFileSpecifyDeviceObjectHint (<no parameter info>)
fffff805`32be8ed0 nt!VfZwCreateFile (<no parameter info>)
fffff805`32be76d0 nt!VerifierNtCreateFile (<no parameter info>)
fffff805`32f420a8 nt!pXdvIoCreateFileEx = <no type information>
fffff805`32f4a8d8 nt!pXdvZwCreateFile = <no type information>
fffff805`3289f540 nt!IoCreateFileSpecifyDeviceObjectHint (<no parameter info>)
fffff805`32bd2840 nt!VerifierIoCreateFileEx (<no parameter info>)
fffff805`32f4a6b0 nt!pXdvNtCreateFile = <no type information>
fffff805`3280b920 nt!IopCreateFile (<no parameter info>)
fffff805`3280b630 nt!IoCreateFile (<no parameter info>)
fffff805`32f4aec8 nt!pXdvIoCreateFile = <no type information>
fffff805`32bb6ae0 nt!KdpCreateFileCallback (<no parameter info>)
fffff805`32bd2790 nt!VerifierIoCreateFile (<no parameter info>)

```

Xuất hiện:

nt!NtCreateFile

Giải thích

Đây là hàm hệ thống chịu trách nhiệm tạo hoặc mở file trong Windows Kernel.

Bước 7: Disassemble hàm NtCreateFile

Thực hiện:

u nt!NtCreateFile

```

kd> u nt!NtCreateFile
nt!NtCreateFile:
fffff805`3280b890 4881ec8800000000 sub     rsp,88h
fffff805`3280b897 33c0    xor     eax,eax
fffff805`3280b899 4889442478 mov     qword ptr [rsp+78h],rax
fffff805`3280b89e c744247020000000 mov     dword ptr [rsp+70h],20h
fffff805`3280b8a6 89442468 mov     dword ptr [rsp+68h],eax
fffff805`3280b8aa 4889442460 mov     qword ptr [rsp+60h],rax
fffff805`3280b8af 89442458 mov     dword ptr [rsp+58h],eax
fffff805`3280b8b3 8b8424e000000000 mov     eax,dword ptr [rsp+0E0h]
kd>

```

Mô tả

Lệnh:

u

(Unassemble)

chuyển mã máy thành mã Assembly.

Nhận xét

Có thể quan sát các lệnh CPU thực thi khi hàm NtCreateFile được gọi.

Bước 8: Sử dụng cửa sổ Disassembly

Mở:

View

→ Disassembly

Nhập:

nt!NtCreateFile

```
kd> uf nt!NtCreateFile
nt!NtCreateFile:
fffff805`3280b890 4881ec88000000 sub     rsp,88h
fffff805`3280b897 33c0          xor     eax,eax
fffff805`3280b899 4889442478     mov     qword ptr [rsp+78h],rax
fffff805`3280b89e c744247020000000 mov     dword ptr [rsp+70h],20h
fffff805`3280b8a6 89442468       mov     dword ptr [rsp+68h],eax
fffff805`3280b8aa 4889442460     mov     qword ptr [rsp+60h],rax
fffff805`3280b8af 89442458       mov     dword ptr [rsp+58h],eax
fffff805`3280b8b3 8b8424e0000000 mov     eax,dword ptr [rsp+0E0h]
fffff805`3280b8ba 89442450       mov     dword ptr [rsp+50h],eax
fffff805`3280b8be 488b8424d8000000 mov     rax,qword ptr [rsp+0D8h]
fffff805`3280b8c6 4889442448     mov     qword ptr [rsp+48h],rax
fffff805`3280b8cb 8b8424d0000000 mov     eax,dword ptr [rsp+0D0h]
fffff805`3280b8d2 89442440       mov     dword ptr [rsp+40h],eax
fffff805`3280b8d6 8b8424c8000000 mov     eax,dword ptr [rsp+0C8h]
fffff805`3280b8dd 89442438       mov     dword ptr [rsp+38h],eax
fffff805`3280b8e1 8b8424c0000000 mov     eax,dword ptr [rsp+0C0h]
fffff805`3280b8e8 89442430       mov     dword ptr [rsp+30h],eax
fffff805`3280b8ec 8b8424b8000000 mov     eax,dword ptr [rsp+0B8h]
fffff805`3280b8f3 89442428       mov     dword ptr [rsp+28h],eax
fffff805`3280b8f7 488b8424b0000000 mov     rax,qword ptr [rsp+0B0h]
fffff805`3280b8ff 4889442420     mov     qword ptr [rsp+20h],rax
fffff805`3280b904 e817000000     call    nt!IopCreateFile (fffff805`3280b920)
fffff805`3280b909 4881c488000000 add     rsp,88h
fffff805`3280b910 c3           ret
```

Sau đó:

nt!NtCreateFile+16

```

kd> u nt!NtCreateFile L30
nt!NtCreateFile:
fffff805`3280b890 4881ec88000000 sub     rsp,88h
fffff805`3280b897 33c0          xor     eax,eax
fffff805`3280b899 4889442478    mov     qword ptr [rsp+78h],rax
fffff805`3280b89e c744247020000000 mov     dword ptr [rsp+70h],20h
fffff805`3280b8a6 89442468      mov     dword ptr [rsp+68h],eax
fffff805`3280b8aa 4889442460    mov     qword ptr [rsp+60h],rax
fffff805`3280b8af 89442458      mov     dword ptr [rsp+58h],eax
fffff805`3280b8b3 8b8424e0000000 mov     eax,dword ptr [rsp+0E0h]
fffff805`3280b8ba 89442450      mov     dword ptr [rsp+50h],eax
fffff805`3280b8be 488b8424d8000000 mov     rax,qword ptr [rsp+0D8h]
fffff805`3280b8c6 4889442448    mov     qword ptr [rsp+48h],rax
fffff805`3280b8cb 8b8424d0000000 mov     eax,dword ptr [rsp+0D0h]
fffff805`3280b8d2 89442440      mov     dword ptr [rsp+40h],eax
fffff805`3280b8d6 8b8424c8000000 mov     eax,dword ptr [rsp+0C8h]
fffff805`3280b8dd 89442438      mov     dword ptr [rsp+38h],eax
fffff805`3280b8e1 8b8424c0000000 mov     eax,dword ptr [rsp+0C0h]
fffff805`3280b8e8 89442430      mov     dword ptr [rsp+30h],eax
fffff805`3280b8ec 8b8424b8000000 mov     eax,dword ptr [rsp+0B8h]
fffff805`3280b8f3 89442428      mov     dword ptr [rsp+28h],eax
fffff805`3280b8f7 488b8424b0000000 mov     rax,qword ptr [rsp+0B0h]
fffff805`3280b8ff 4889442420    mov     qword ptr [rsp+20h],rax
fffff805`3280b904 e817000000    call    nt!IopCreateFile (fffff805`3280b920)
fffff805`3280b909 4881c488000000 add     rsp,88h
fffff805`3280b910 c3           ret
fffff805`3280b911 cc           int     3
fffff805`3280b912 cc           int     3
fffff805`3280b913 cc           int     3
fffff805`3280b914 cc           int     3
fffff805`3280b915 cc           int     3
fffff805`3280b916 cc           int     3
fffff805`3280b917 cc           int     3
fffff805`3280b918 cc           int     3
fffff805`3280b919 cc           int     3
fffff805`3280b91a cc           int     3
fffff805`3280b91b cc           int     3
fffff805`3280b91c cc           int     3
fffff805`3280b91d cc           int     3
fffff805`3280b91e cc           int     3
fffff805`3280b91f cc           int     3
nt!IopCreateFile:
fffff805`3280b920 488bc4        mov     rax,rsp
fffff805`3280b923 4c894820      mov     qword ptr [rax+20h],r9
fffff805`3280b927 4c894018      mov     qword ptr [rax+18h],r8
fffff805`3280b92b 895010        mov     dword ptr [rax+10h],edx
fffff805`3280b92e 48894808      mov     qword ptr [rax+8],rcx
fffff805`3280b932 53           push    rbx
fffff805`3280b933 56           push    rsi
fffff805`3280b934 57           push    rdi
fffff805`3280b935 4154         push    r12

```

Bước 9: Trợ giúp của WinDbg

Thực hiện:

?

```
kd> ?  
  
Open debugger.chm for complete debugger documentation  
  
B[C|D|E][<bps>] - clear/disable/enable breakpoint(s)  
BL - list breakpoints  
BA <access> <size> <addr> - set processor breakpoint  
BP <address> - set soft breakpoint  
D[type][<range>] - dump memory  
DT [-n|y] [[mod!]name] [[-n|y]fields]  
    [address] [-l list] [-a[]|c|i|o|r[#]|v] - dump using type information  
DV [<name>] - dump local variables  
DX [-r[#]] <expr> - display C++ expression using extension model (e.g.: NatVis)  
E[type] <address> [<values>] - enter memory values  
G[H|N] [=<address> [<address>...]] - go  
K <count> - stacktrace  
KP <count> - stacktrace with source arguments  
LM[k|l|u|v] - list modules  
LN <expr> - list nearest symbols  
P [=<addr>] [<value>] - step over  
Q - quit  
R [[<reg> [= <expr>]]] - view or set registers  
S[<opts>] <range> <values> - search memory  
SX [{e|d|i|n} [-c "Cmd1"] [-c2 "Cmd2"] [-h] {Exception|Event|*}] - event filter  
T [=<address>] [<expr>] - trace into  
U [<range>] - unassemble  
version - show debuggee and debugger version  
X [<[*|module>!]<[*|symbol>] - view symbols
```

Mô tả

Hiển thị hệ thống trợ giúp tích hợp của WinDbg.

Nhận xét

Cho phép tra cứu nhanh các lệnh debug.

Bước 10: Xem cấu trúc DRIVER_OBJECT

Thực hiện:

dt nt!_DRIVER_OBJECT

```
kd> dt nt!_DRIVER_OBJECT
+0x070 MajorFunction : [28] Ptr64 long
+0x000 Type : Int2B
+0x002 Size : Int2B
+0x008 DeviceObject : Ptr64 _DEVICE_OBJECT
+0x010 Flags : Uint4B
+0x018 DriverStart : Ptr64 Void
+0x020 DriverSize : Uint4B
+0x028 DriverSection : Ptr64 Void
+0x030 DriverExtension : Ptr64 _DRIVER_EXTENSION
+0x038 DriverName : _UNICODE_STRING
+0x048 HardwareDatabase : Ptr64 _UNICODE_STRING
+0x050 FastIoDispatch : Ptr64 _FAST_IO_DISPATCH
+0x058 DriverInit : Ptr64 long
+0x060 DriverStartIo : Ptr64 void
+0x068 DriverUnload : Ptr64 void
+0x070 MajorFunction : [28] Ptr64 long
kd>
```

Mô tả

Lệnh **dt** (Display Type) hiển thị cấu trúc dữ liệu của một kiểu dữ liệu trong kernel.

Kết quả

Quan sát trường:

DriverStart

Giải thích

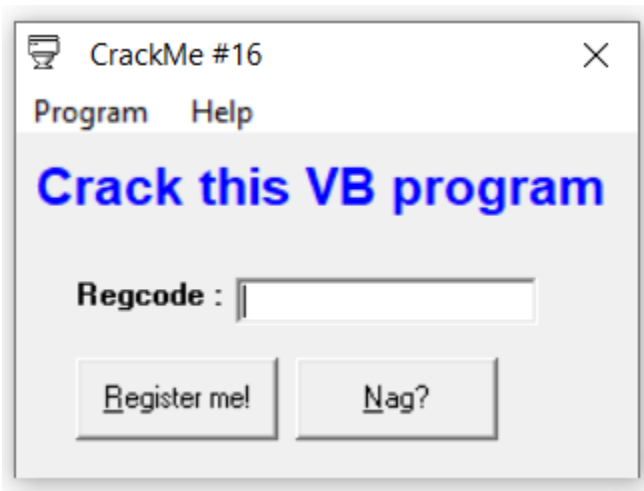
DriverStart chứa địa chỉ bắt đầu của driver trong bộ nhớ.

Kết luận

Trong bài thực hành này em đã sử dụng thành công WinDbg và LiveKD để phân tích hệ thống Windows ở mức kernel. Em đã thực hiện các lệnh liệt kê module, xem nội dung bộ nhớ, tìm kiếm symbol, disassemble hàm NtCreateFile và quan sát cấu trúc DRIVER_OBJECT. Qua đó hiểu rõ hơn về cơ chế hoạt động của Windows Kernel cũng như các kỹ thuật phân tích hệ thống bằng WinDbg.

Crack me 16 Lê Trọng Tấn HE194508

This crackme is created with Visual Basic 5/6 which is prior to the .NET framework. Some programs out there are still written in it. So here's your opportunity to crack such a program. It is a mod of one of Lena's VB programs. Note that this crackme is compiled to native format. The p-code format will be covered in another crackme.



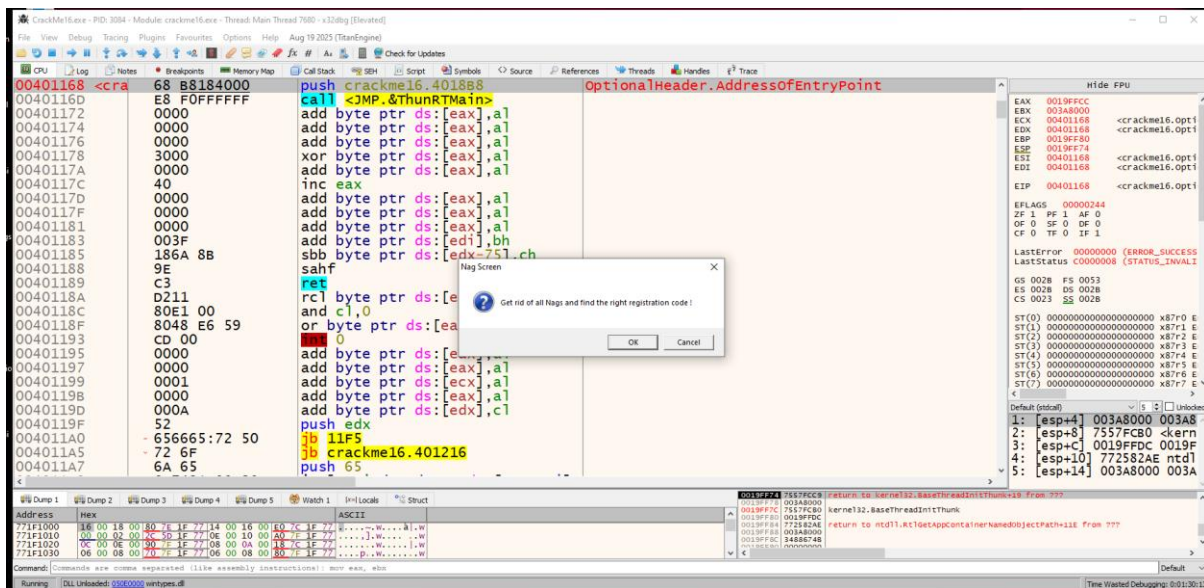
There are 2 tasks:

1. Get rid of the Nag screen which pops-up before the above window shows.
2. Crack the Regcode.

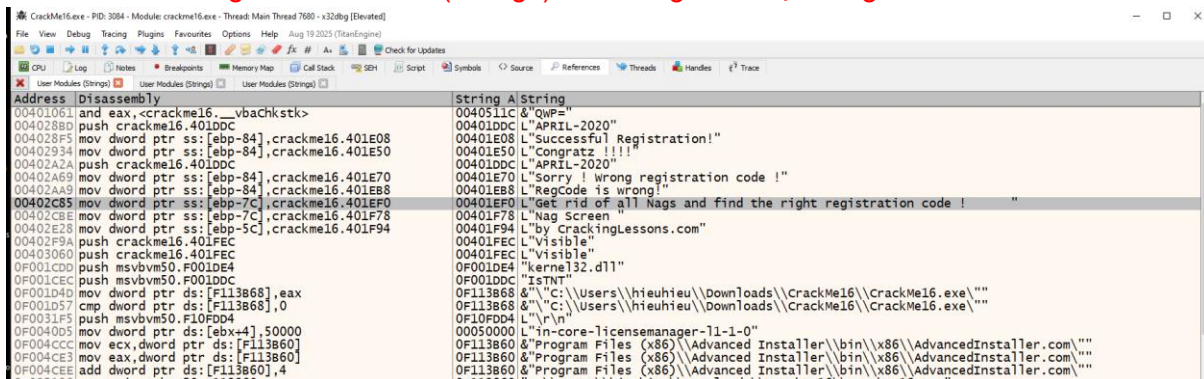
Bài làm

1.

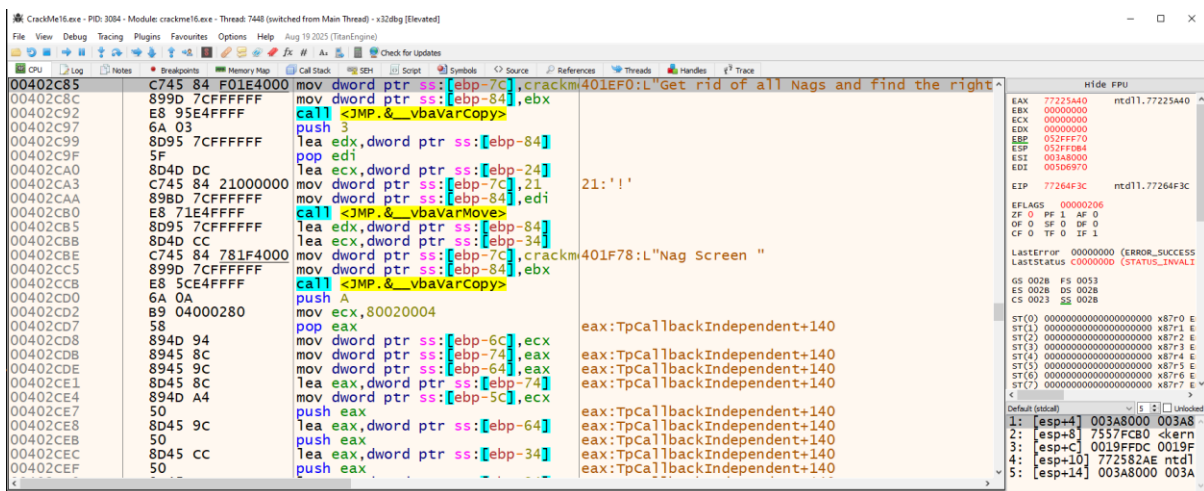
Em chạy chương trình, có Nag Screen xuất hiện, bây giờ ta sẽ remove Nag Screen này



Em tìm kiếm trong User Modules (Strings) tìm String hiển thị ở Nag Screen



Click vào String, Em được đưa đến Address 00402C85:



Lướt xuống tầm 20 dòng, em thấy tại address 00402CFE là lệnh gọi đến hàm hiển thị hộp thoại Hàm rtcMsgBox tạo và hiển thị cửa sổ Nag Screen

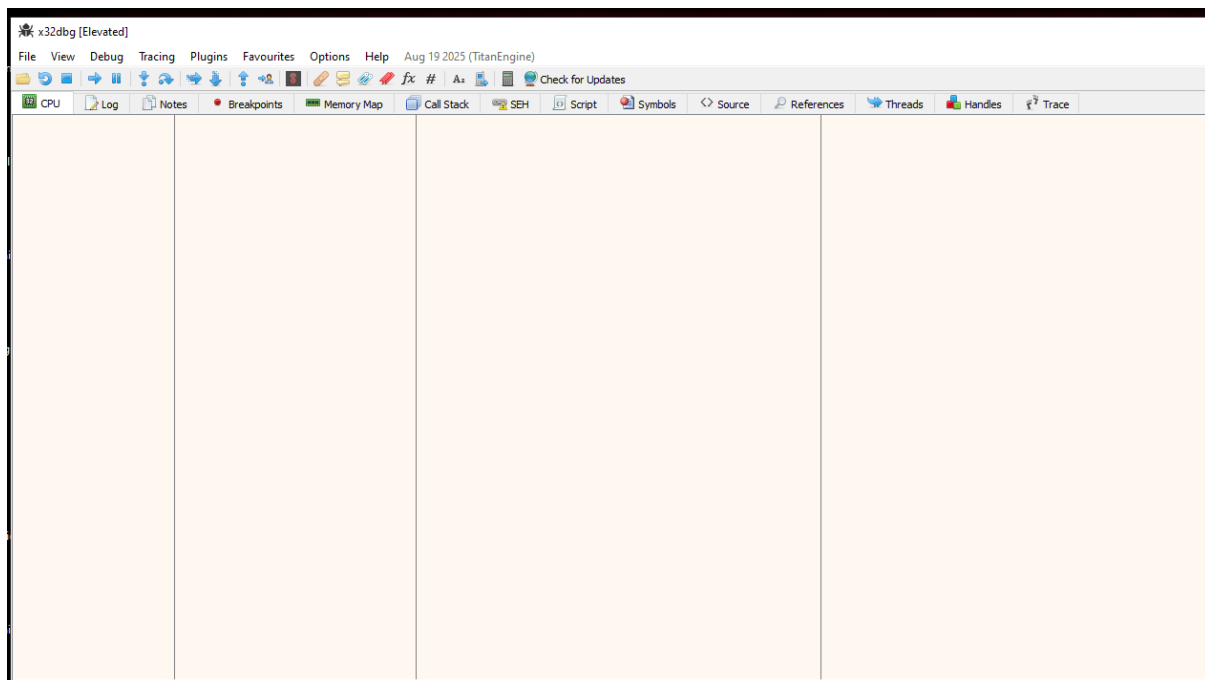
00402CEB	50	push eax	
00402CEC	8D45 CC	lea eax,dword ptr ss:[ebp-34]	
00402CEF	50	push eax	
00402CF0	8D45 DC	lea eax,dword ptr ss:[ebp-24]	
00402CF3	50	push eax	
00402CF4	E8 21E4FFFF	call <JMP.&_vbaI4Var>	
00402CF9	50	push eax	
00402CFA	8D45 AC	lea eax,dword ptr ss:[ebp-54]	
00402CFD	50	push eax	
00402CFE	E8 1DE4FFFF	call <JMP.&rtcMsgBox>	
00402D03	8D95 5CFFFFFF	lea edx,dword ptr ss:[ebp-A4]	ecx:ZWAssociateWaitCompletionPacket+C
00402D09	8D4D BC	lea ecx,dword ptr ss:[ebp-44]	
00402D0C	8985 64FFFFFF	mov dword ptr ss:[ebp-9C],eax	
00402D12	89BD 5CFFFFFF	mov dword ptr ss:[ebp-A4],edi	
00402D18	E8 09E4FFFF	call <JMP.&_vbaVarMove>	
00402D1D	8D45 8C	lea eax,dword ptr ss:[ebp-74]	
00402D20	50	push eax	
00402D21	8D45 9C	lea eax,dword ptr ss:[ebp-64]	

Sau đó chuột phải vào địa chỉ **00402D03** , chọn Assemble và sửa thành nop, sửa thành nop cho 4 địa chỉ liền sau nữa,NOP (No Operation) là lệnh không làm gì cả. CPU chỉ đọc lệnh, tăng EIP sang lệnh tiếp theo, không thay đổi thanh ghi hay bộ nhớ.Sau khi sửa thì CPU sẽ không gọi rtcMsgBox, chạy tiếp mov eax,[ebp-44], làm Nag không xuất hiện

00402CF3	50	push eax	eax:TpCallbackIndependent+140
00402CF4	E8 21E4FFFF	call <JMP.&_vbaI4Var>	
00402CF9	50	push eax	eax:TpCallbackIndependent+140
00402CFA	8D45 AC	lea eax,dword ptr ss:[ebp-54]	eax:TpCallbackIndependent+140
00402CFD	50	push eax	eax:TpCallbackIndependent+140
00402CFE	90	nop	
00402CFF	90	nop	
00402D00	90	nop	
00402D01	90	nop	
00402D02	90	nop	
00402D03	8D95 5CFFFFFF	lea edx,dword ptr ss:[ebp-A4]	
00402D09	8D4D BC	lea ecx,dword ptr ss:[ebp-44]	
00402D0C	8985 64FFFFFF	mov dword ptr ss:[ebp-9C],eax	eax:TpCallbackIndependent+140
00402D12	89BD 5CFFFFFF	mov dword ptr ss:[ebp-A4],edi	
00402D18	E8 09E4FFFF	call <JMP.&_vbaVarMove>	
00402D1D	8D45 8C	lea eax,dword ptr ss:[ebp-74]	eax:TpCallbackIndependent+140
00402D20	50	push eax	eax:TpCallbackIndependent+140
00402D21	8D45 9C	lea eax,dword ptr ss:[ebp-64]	eax:TpCallbackIndependent+140
00402D24	50	push eax	eax:TpCallbackIndependent+140
00402D25	6A 02	push 2	

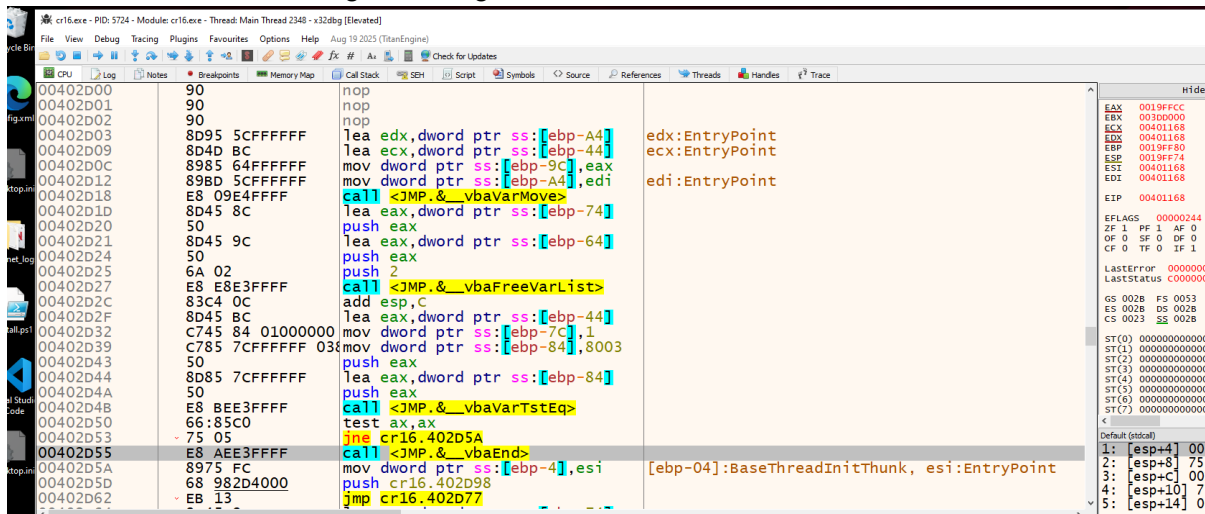
The screenshot shows the OllyDbg interface with the assembly window displaying code at address 00402D03. The code is being patched to NOP. The patching window shows a list of patches for 'crackme16.exe'.

Address	Old Instruction	New Instruction
0100402CFE:EE->90	call <JMP.&_vbaI4Var>	nop
0100402CFF:1D->90	lea eax,dword ptr ss:[ebp-54]	nop
0100402D01:EE->90	push eax	nop
0100402D02:FF->90	push eax	nop



Sau khi patches file và chạy lại, chương trình đã đóng lại và quit, chứng tỏ có lỗi ở đâu đó

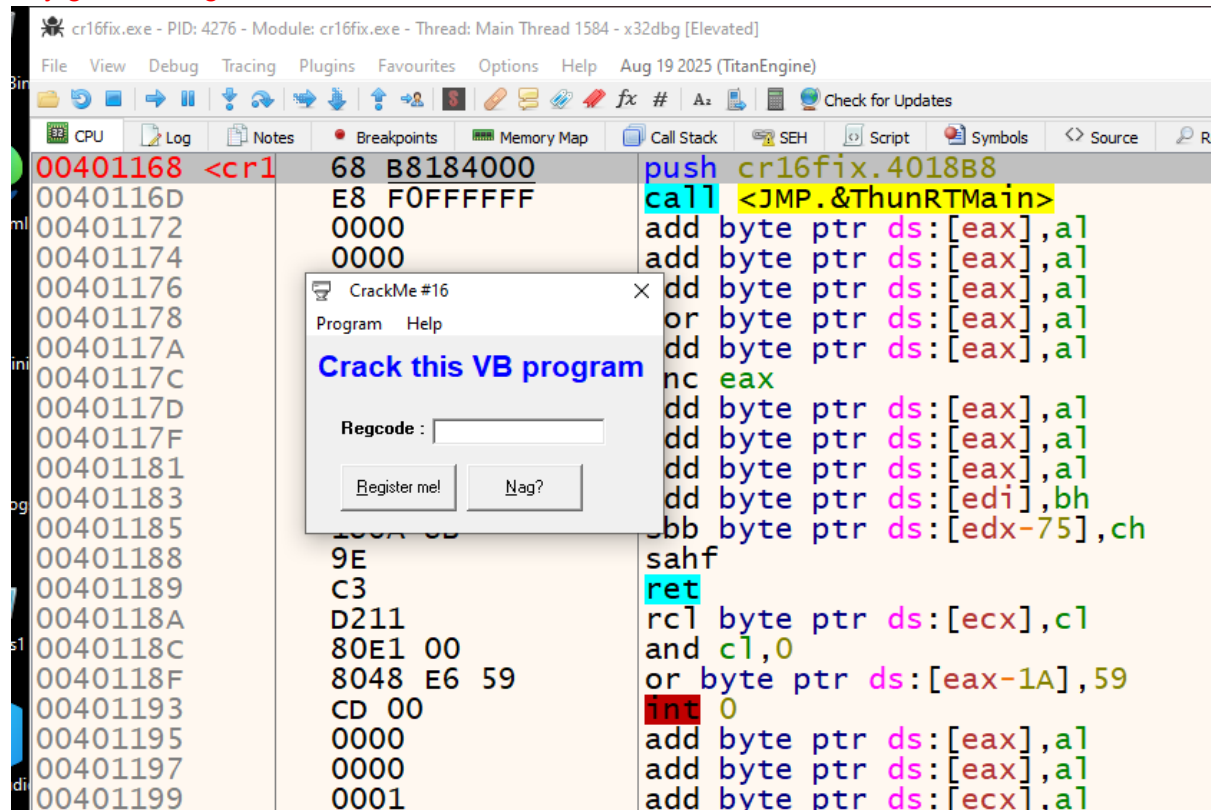
Mở lại đoạn Em đã điền nop, Em thấy chương trình chạy đến dòng **00402D55** và đóng lại, vì thế nên ta sẽ làm cho dòng đó không được thực thi



00402D50	66:85C0	test ax,ax	
00402D53	75 05	jne cr16.402D5A	
00402D55	E8 AEE3FFFF	call <JMP.&__vbaEnd>	
00402D5A	8975 FC	mov dword ptr ss:[ebp-4],esi	[ebp-04]:
00402D5D	68 982D4000	push cr16.402D98	
00402D50	66:85C0	test ax,ax	
00402D53	EB 05	jmp cr16.402D5A	
00402D55	E8 AEE3FFFF	call <JMP.&__vbaEnd>	
00402D5A	8975 FC	mov dword ptr ss:[ebp-4],esi	[ebp-04]:Bas

Tại địa chỉ **00402D53**, đổi **jne** thành **jmp** để skip câu lệnh đóng chương trình

Sau đó patches file, chạy chương trình đã patches, thì chương trình đã bỏ qua Nag Screen, bây giờ tìm Regcode



Em tìm trong User Modules (Strings) thì thấy RegCode ở đây là “ APRIL-2020”

